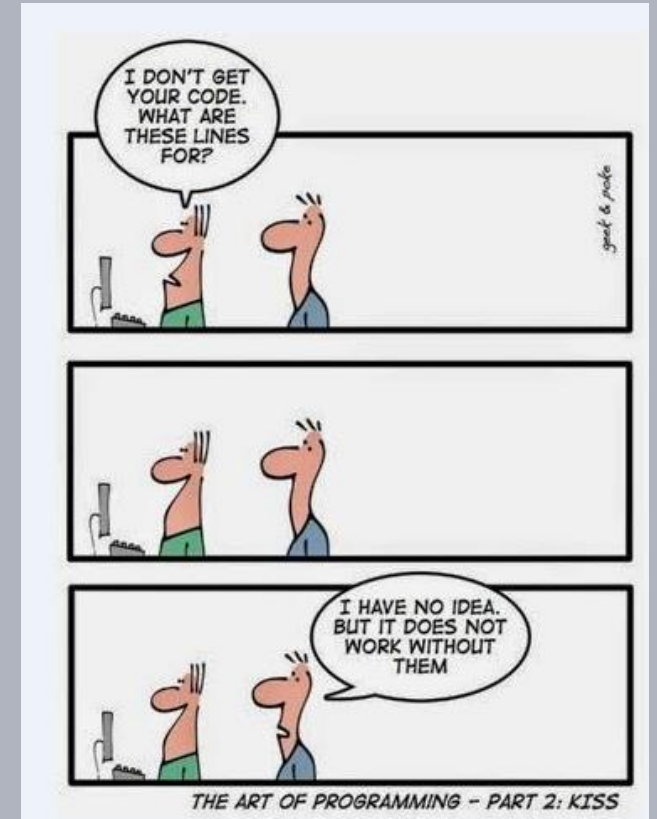


Advanced Software Design Techniques

UML, Inheritance, Polymorphism

Prof. Dr.-Ing. Peter Fromm



Content

- Idea of generalisation and specialisation
- Inheritance
- Polymorphism

Design principle

Code once, use often (= **reuse**)

- Economic way of coding
- Use of tested code
- No confusion where to apply the change



They are both cars...

But there might be some (minor) differences

- Color
- MaxSpeed
- Features of the infotainment system
- ...



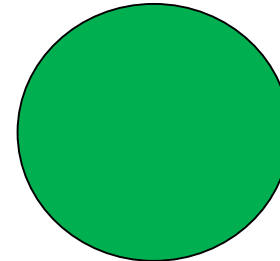
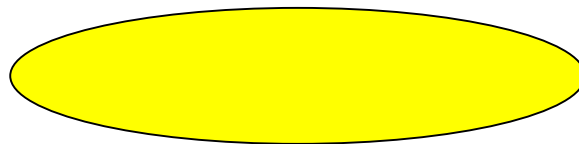
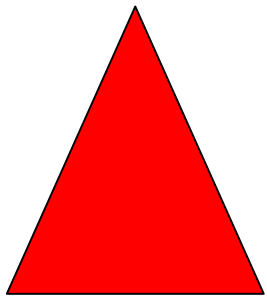
Let's develop a small graphic library

- We want to create different graphical objects
- Store them in a database
- And show them in different windows

Consider the following example:

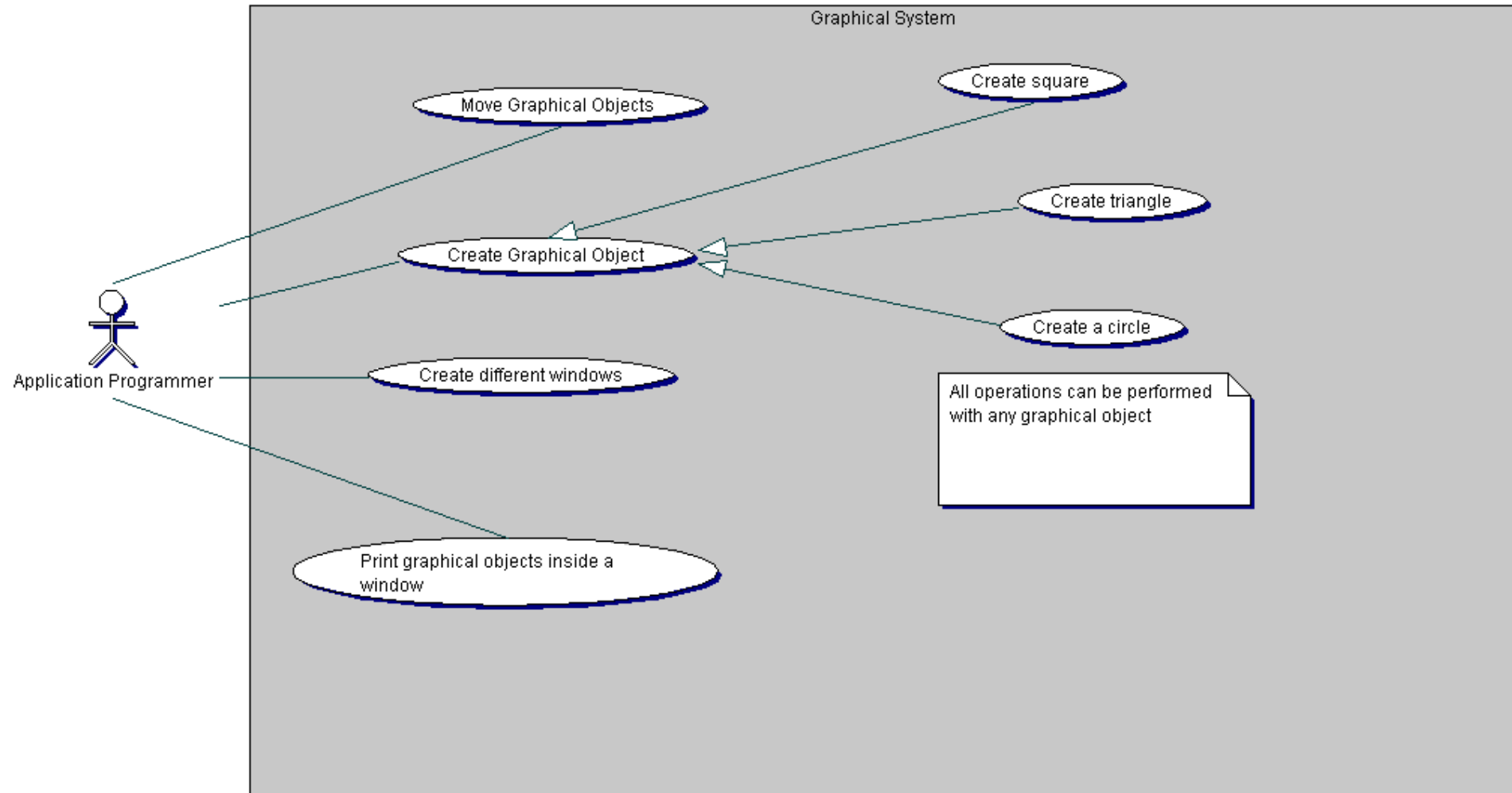
- Which attributes
- Which operations

do they have in common?



Textobject

A new Diagram: Use Case Diagram



Definition Inheritance

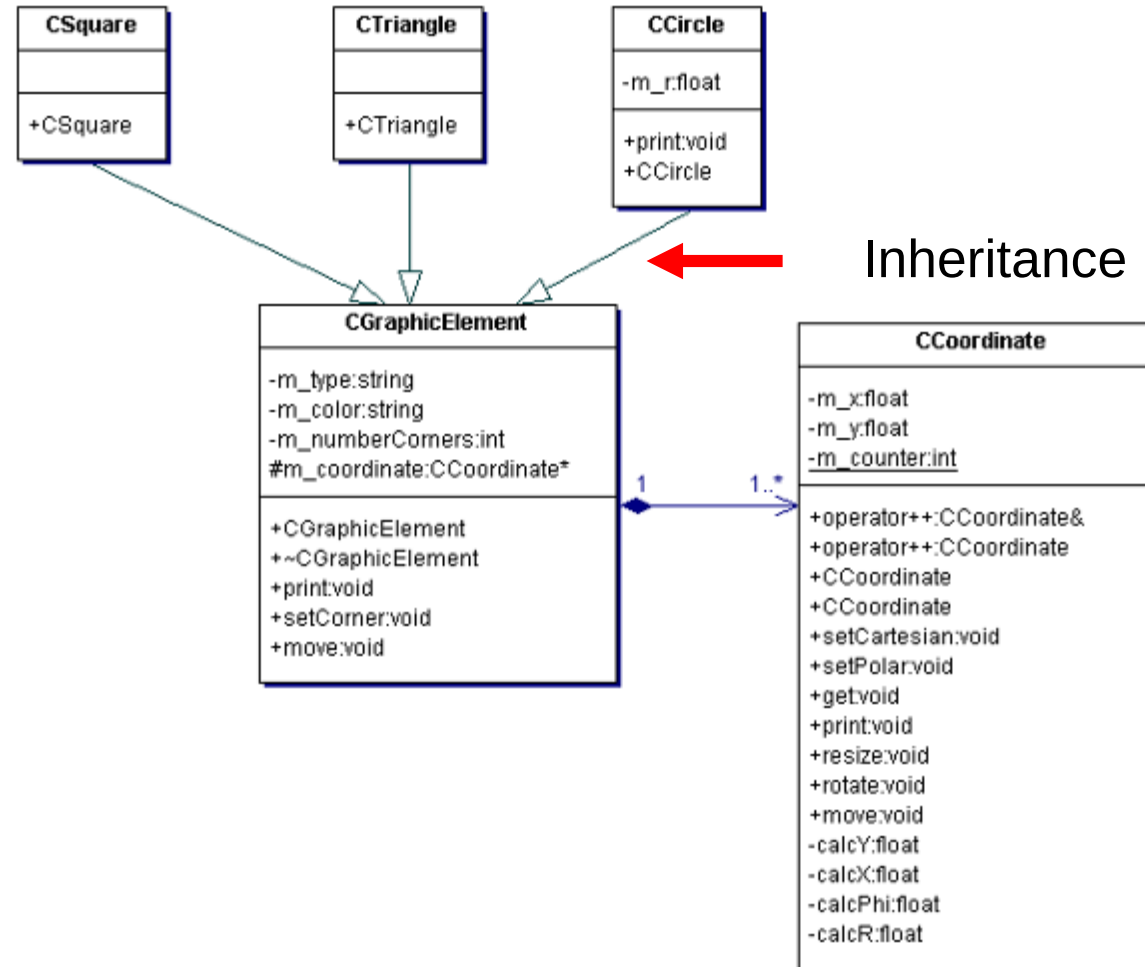
Definition

In object-oriented programming, inheritance is a way to form new classes using classes that have already been defined. Inheritance is intended to help reuse existing code with little or no modification. The new classes, known as derived classes, inherit attributes and behavior of the pre-existing classes, which are referred to as base classes (or ancestor classes). The inheritance relationship of sub- and superclasses gives rise to a hierarchy. The inheritance concept was invented in 1967 for Simula.

(Wikipedia)

Inheritance is used to implement common attributes and operations of several classes in a shared class, the base class.

A first class diagram of a subset of the graphic system



Inheritance in Code

```
#include "CGraphicElement.h"

class CCircle : public CGraphicElement {
private:
    float m_r;
public:

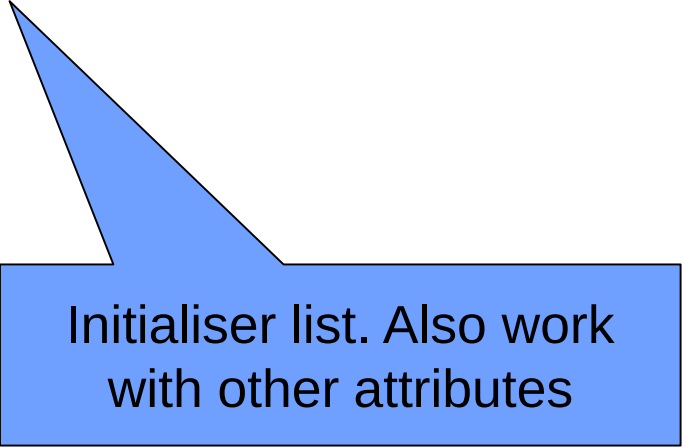
    void print(); //without the own print, the radius would not be printed

    CCircle(float x=0, float y=0, float r=0,
            string color="DefaultColorCircle");
};
```

Most of the time, we use public inheritance, because we want to make all public entities of the base class visible. Sometimes, private inheritance might be useful to hide features of the base class.

Initialisation of the derived class

```
CCircle::CCircle(float x, float y,  
                float r, string color) : CGraphicElement(1,color,"Circle")  
{  
    //Calling a member of the derived class for initialisation  
    setCorner(0,x,y);  
  
    //Setting own attribute  
    m_r = r;  
}
```



Initialiser list. Also work
with other attributes

Overriding inherited methods

Imagine:

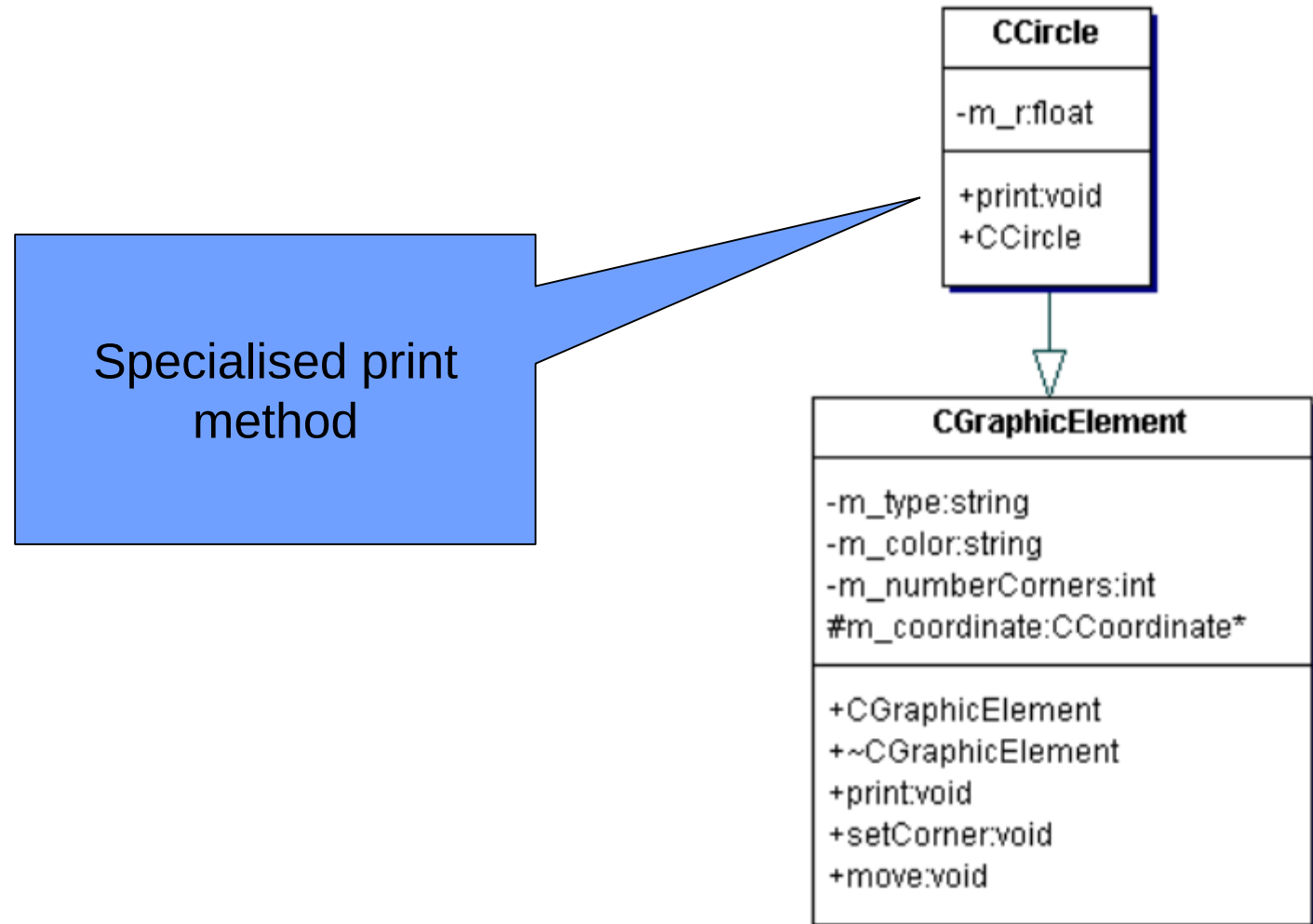
- You have successfully derived a class from a base class
- Everything was fine
- And then you realize that one of the derived functions does not behave like you expected

- Do I have to throw the base class away?
- Do I have to spoil the code by using some terrible preprocessor commands or complex if...then...else structures?

- ➔ C++ provides the solution of overriding methods by simply defining a function with the same name in the derived class!
- ➔ Excellent technique for specialization

Overriding of inherited methods

- The specialised class simply has to add a method with the same name and a different behaviour and we are done.



Now let's create a database...

The following code creates invalid data in the objects. Why?

```
m_square[0] = CSquare("My first square", 1, 1, 2, 3);  
m_circle[0] = CCircle("My first circle", 10, 4, 4);  
m_circle[1] = CCircle ("My 2 circle", 20, 4, 4);  
m_circle[2] = CCircle ("My 3 circle", 30, 4, 4);
```



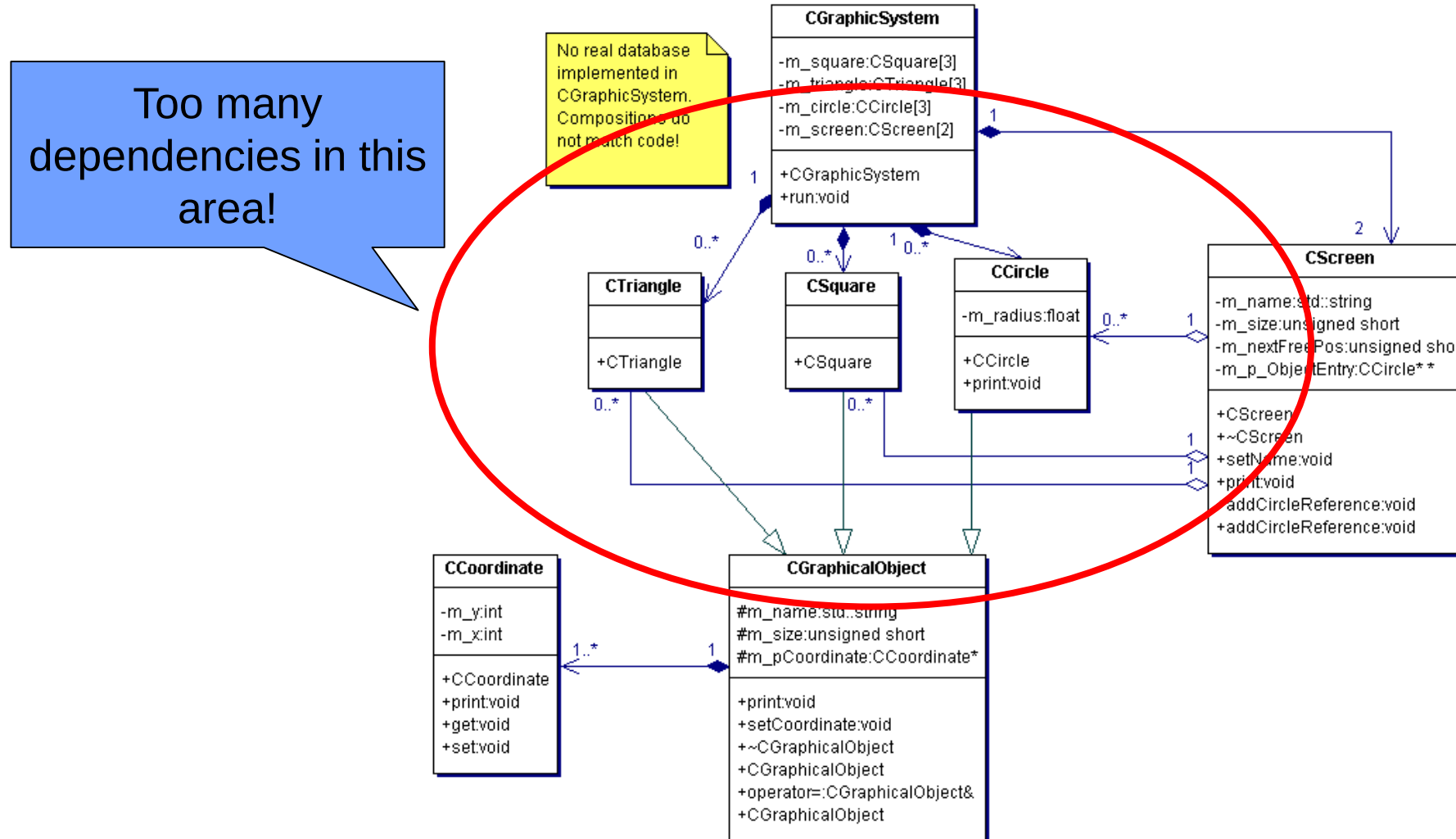
The reason is:

- In CGraphicObject, we create a dynamic array of CCoordinates
- When assigning a temporary object Csquare(...) to m_square[0], the following happens
 - The temporary object is created
 - The POINTER (not the content) is assigned
 - Then the content of the temporary object is deleted.

We need to understand the difference between shallow and deep copies!

And we have to implement copy constructors and assignment operators for all classes containing allocated memory!

Still somewhat messy...



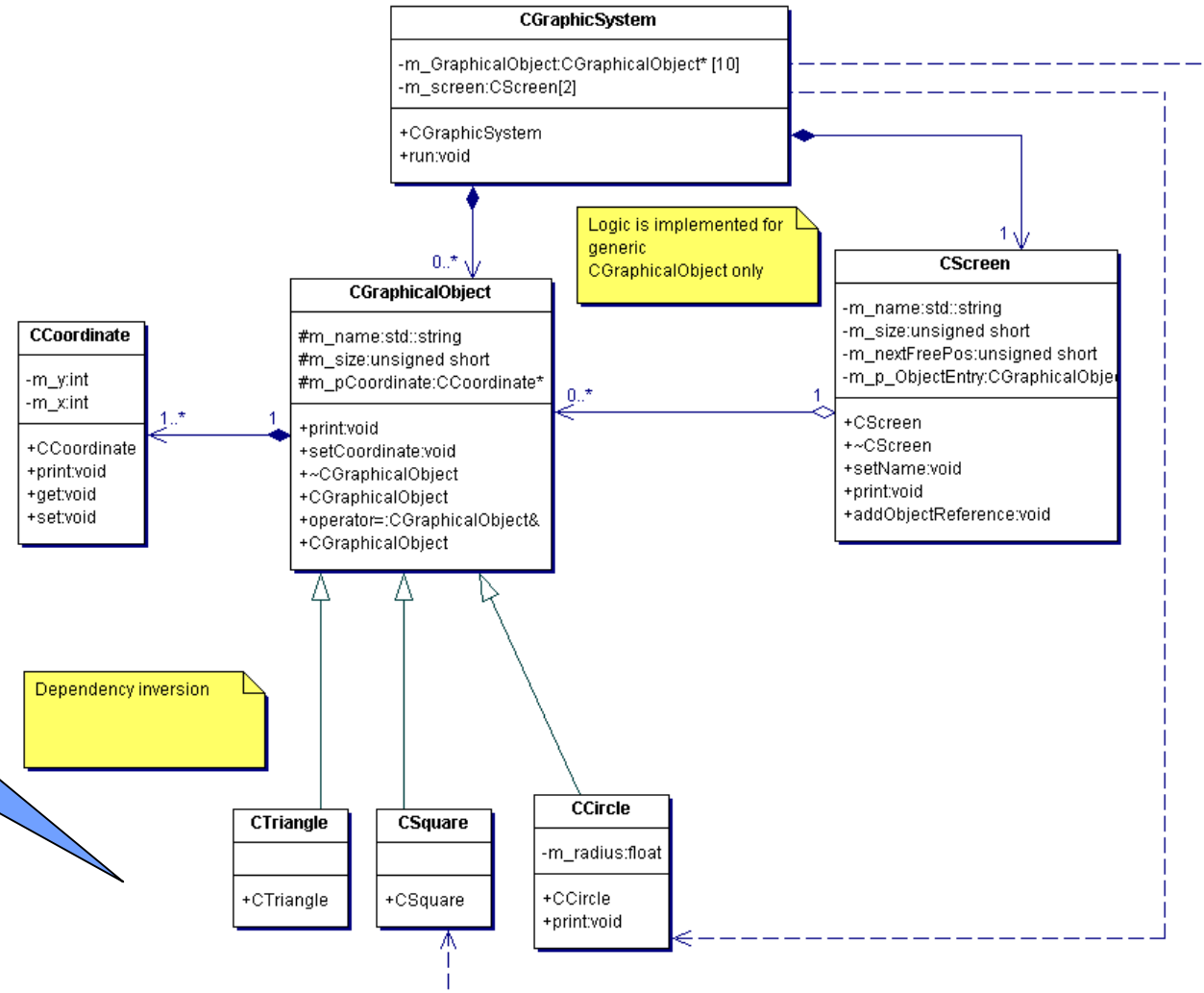
Key idea for design improvement

- The screen is capable of storing ANY kind of object pointer (technically no problem, because every pointer has the same size)
- Whenever calling a method of the dereferenced object pointer, the object KNOWS which type it is and calls the correct method.

Polymorphism

- In general, **polymorphism** describes multiple possible states for a single property (it is said to be *polymorphic* , or *polymorphous*).
- In computer science, **polymorphism** is a programming language feature that allows **values of different data types** to be handled using a **uniform interface** .
- Dynamic polymorphism is also referred to as **Late Binding** . Versus early binding, which is performed by the Linker during the build process

This looks much more cleaned up



Easy extension
of more
specialised
classes

Virtual methods

Virtual

- `virtual void draw(); // virtual`
- Behavior similar to overridden functions in an inheritance relationship
- Mandatory for polymorphism

Pure Virtual

- `virtual void draw() =0; // pure virtual`
- No implementation
- Derived objects must provide a non virtual implementation
- Used e.g. for interface design

Difference between early and late binding

Early binding

```
class CBase
{
    void foo();
}

class CDerived : public CBase
{
    void foo();
}

CDerived d;
d.foo();
```



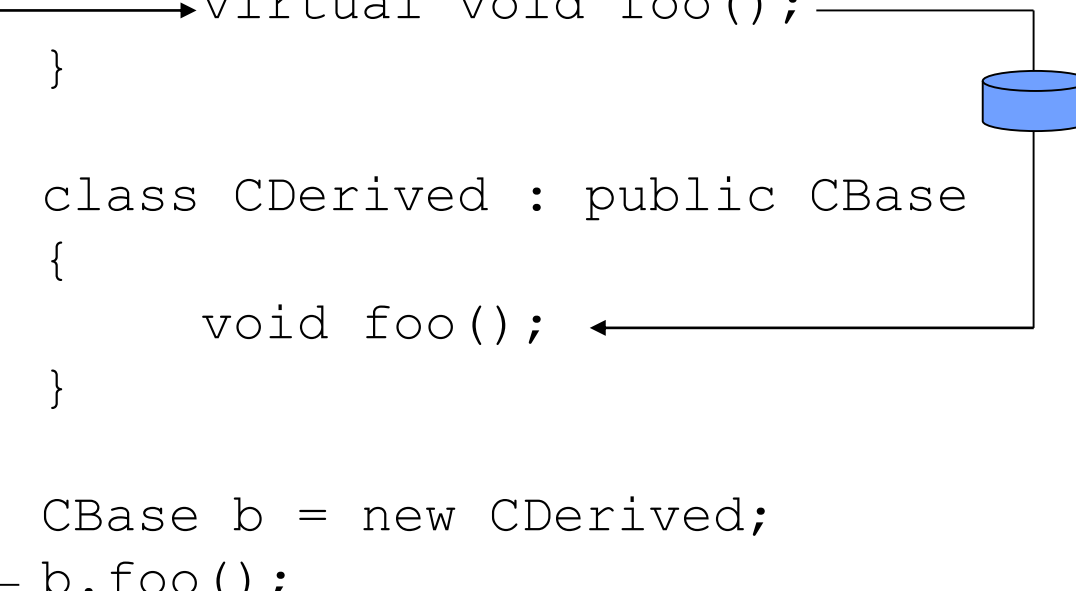
Linker will set
address during
compile time

Late binding

```
class CBase
{
    virtual void foo();
}

class CDerived : public CBase
{
    void foo();
}

CBase b = new CDerived;
b.foo();
```



Function pointer
set during runtime

Polymorphism – some considerations

Polymorphism allows to write very elegant code...

Pitfall Garbage collection

- `baseClass element;`
- `element = new(derivedClass);`
- `delete element;` `// ➔` only the memory from the base class is released. Define the destructor as virtual!

Pitfall Overhead

- Every class of a class hierarchy declaring virtual functions requires a vTable
- Every object requires a “hidden attribute” vpointer that point to the proper vtable ➔ 4 bytes overhead. For embedded systems often not acceptable

Wrap-Up

- Use Case diagram
- Inheritance
- Overriding methods
- Class specialisation
- Polymorphism (inverted dependencies)